arXiv:2610.30218v2 [cs.SE] 3 Nov 2026

JANUS: Agentic Cross-Language Fuzzing Harness Generation for Android Native Libraries

XIANGYU ZHANG, Nankai University, China and Zhongguancun Academy, China
YUCHENG SU, Xiaohongshu Inc., China
LINGLING FAN✉, Nankai University, China and Zhongguancun Academy, China
YINAN LI, Nankai University, China
SEN CHEN, Nankai University, China

Modern Android apps extensively rely on native libraries, bridging Java/Kotlin and C/C++ through the Java Native Interface (JNI). This cross-language interoperation, however, exposes the memory-safe Android Runtime to the unsafe C/C++ world, creating an attack surface that has been repeatedly exploited for full device compromise. While fuzzing is a proven technique for vetting native libraries, state-of-the-art harness generation tools struggle to faithfully capture closed-source cross-language API usage: (*i*) precise program analysis across the JNI boundary does not scale, whereas lightweight approximations miss critical invocation patterns; and (*ii*) they fall short in constructing complex composite objects consumed by JNI methods.

In this paper, we propose JANUS, the first agentic approach to synthesizing cross-language JNI-aware fuzzing harnesses for Android native libraries. JANUS iteratively resolves both explicit and implicit cross-language call dependencies to recover app-specific runtime context, and recursively assembles cross-language argument objects in a bottom-up manner. JANUS leverages an autonomous agentic workflow to reduce reliance on hand-crafted heuristics, capturing the vast landscape of hidden API usage knowledge embedded in closed-source native libraries. Evaluation on 44 popular real-world apps (such as Google and YouTube Music, covering 21,386 JNI methods total) shows that JANUS produces over 5.9× more valid harnesses (6,094) than state-of-the-art automatic harnessing tools. On pre-installed apps from four mainstream Android vendors (e.g., Xiaomi, Vivo), JANUS further uncovers 6 previously unknown vulnerabilities in closed-source native libraries.

CCS Concepts: • **Security and privacy → Software and application security**.

Additional Key Words and Phrases: Android Native Libraries, Harness Generation, LLM Agent

ACM Reference Format:
Xiangyu Zhang, Yucheng Su, Lingling Fan, Yinan Li, and Sen Chen. 2026. JANUS: Agentic Cross-Language Fuzzing Harness Generation for Android Native Libraries. *ACM Trans. Softw. Eng. Methodol.* , (2026), 27 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Native libraries are pervasive in modern Android apps, invoked from Java/Kotlin through the Java Native Interface (JNI) for performance gains or to reuse battle-tested legacy code. This convenience, however, comes at a steep security cost:

Authors' Contact Information: Xiangyu Zhang, DISSec, NDST, College of Cryptology and Cyber Science, Nankai University, Tianjin, China and Zhongguancun Academy, Beijing, China; Yucheng Su, Xiaohongshu Inc., Beijing, China; Lingling Fan, linglingfan@nankai.edu.cn, DISSec, NDST, College of Cryptology and Cyber Science, Nankai University, Tianjin, China and Zhongguancun Academy, Beijing, China; Yinan Li, DISSec, NDST, College of Cryptology and Cyber Science, Nankai University, Tianjin, China; Sen Chen, DISSec, NDST, College of Cryptology and Cyber Science, Nankai University, Tianjin, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

the JNI bridge re-stitches the memory-safe Android Runtime (ART) onto the unsafe C/C++ world; once loaded, a native library shares the hosting process's address space and executes under the same user ID (UID) and granted permissions, yet its memory accesses are *not* protected by ART's managed-memory safety guarantees [6]. Memory-corruption defects in native libraries thus reintroduce a cross-language attack surface into otherwise sandboxed processes, and have repeatedly served as launchpads for full device compromise. Most recently, this attack surface has been notably weaponized by the advanced spyware LANDFALL [62] and by the 0-click chain that Google Project Zero used to break Pixel 10 [22]. Thus, **given how deeply native code is woven into today's Android ecosystem, the need for scalable approaches to vetting this fast-growing cross-language attack surface has never been more urgent.**

Android native libraries have been shown by prior work [19, 66] to be effectively auditable through fuzzing. Central to this process is *harness generation*, i.e., synthesizing a wrapper program that performs the necessary initialization and argument construction before feeding fuzzer-generated data to the target API [57]. However, harnessing closed-source Android native libraries presents **two inherent issues** that have not been adequately addressed. (*i*) Recovering cross-language API usage knowledge related to harness synthesis is non-trivial, as it demands knowledge scattered across two languages. Two complementary aspects should be acquired for valid harnesses: *call dependencies* (i.e., the invocation sequence among JNIs) and *argument construction* (i.e., how arguments are built and passed) [12, 25, 66]. (*ii*) Native libraries are predominantly closed-source and privately bound to their host apps. Consequently, JNI usage patterns can be obtained neither from official documentation nor from portable examples across apps. Each app effectively defines its own private API usage conventions, making such knowledge largely non-transferable and requiring it to be recovered anew for each app [19].

In light of this, recent studies [19, 66] attempt to learn JNI usage knowledge and synthesize JNI-aware harnesses automatically. Atlas [66] performs cross-language symbolic execution and taint propagation in pursuit of precise Java↔Native data-dependency recovery, but the inherent path explosion prevents it from scaling to contemporary commercial Android apps (only 51% directly usable in its 17-vendor-app dataset). Another state-of-the-art tool, Poirot [19], takes the opposite stance, dropping native-side analysis in favor of lightweight Java-side data- and control-flow analysis to mimic each app's behavior; however, the Java-only strategy, while more scalable, sacrifices substantial precision (only 49.5% valid across ~4K apps). Both state-of-the-art artifacts share two limitations: (*i*) they struggle to capture cross-language API usage knowledge, facing an inherent precision-scalability trade-off in static analysis across the JNI boundary: precise cross-language analysis is notoriously hard to scale [66], while lightweight approximations tend to miss critical invocation patterns [19]; and (*ii*) they fall short of addressing how to acquire argument-construction knowledge for the complex composite objects routinely consumed by JNI methods.

Root Challenges. At the root of the predicament (prior artifacts are trapped in) lie two fundamental challenges unique to harnessing Android native libraries. **C1** Faithfully reconstructing cross-language invocation sequences requires intricate, app-specific runtime context (e.g., Java-side argument-object instantiation and field assignment, and native-side global state and runtime handles). This process often involves missing or misordered explicit and implicit dependencies, which can readily lead to silent failures or trivially shallow execution. **C2** Cross-language argument construction exposes a non-trivial semantic gap. JNI methods in real-world applications commonly accept complex objects (e.g., nested, user-defined composite types) whose construction logic cannot be inferred from JNI type signatures alone. Since the harness executes on the native side, these Java objects require assembly from scratch through reflection-style JNI functions [34], making it inherently challenging to reconstruct objects and capture the semantic relationship that conform to native-side expectations.

Our Approach. To overcome the inherent challenges above, we propose **Janus**, an agentic approach that synthesizes far-reaching and meaningful cross-language JNI-aware harnesses. Janus's **key insights** lie in two algorithms interacting with dedicated agentic workflows: To tackle C1, Janus handles cross-language invocation sequences by treating unmet dependencies as the driving force, iteratively resolving both explicit and implicit cross-language invocation context, which spans Java-side object instantiation and native-side runtime state. To address C2, Janus recursively decomposes complex composite objects into directly synthesizable leaf types, then reassembles them bottom-up to construct the objects expected by the native side. Janus implements core algorithms within an agentic workflow powered by Large Language Models (LLMs), capturing the JNI usage knowledge and semantic relationships between parameters, reducing reliance on brittle hand-crafted heuristics, thereby empirically mitigating the precision-scalability trade-off faced by prior work.

Agentic Design. Janus builds an agentic workflow tailored for JNI-aware harness synthesis, in which the underlying LLMs act as autonomous agents that perform code reasoning, planning, tool invocation, and self-reflection throughout harness synthesis, rather than serving as auxiliary components in a conventional fuzzing pipeline [24, 73]. This agentic design, however, surfaces three specific challenges: (*i*) aimless exploration from sprawling cross-language code fragments; (*ii*) app-specific runtime context inconsistency over long-horizon reasoning; and (*iii*) hallucinations in JNI-aware harness generation. To sidestep the trap of overwhelming the LLM with sprawling cross-language code fragments and asking it to single out the critical pieces, Janus equips the agent with a tool-augmented, structure-aware code retriever that forestalls aimless exploration. To tame context drift and hallucinations, Janus persists verified intermediate artifacts across cross-language methods in a dedicated memory layer and closes the generation loop with a validation-guided iterative refinement that attributes failures to the responsible synthesis stage for targeted refinement.

Results. We evaluate Janus on 21,386 JNI methods of 44 real-world Android apps against two state-of-the-art harnessing tools for Android native libraries, Atlas++ [10] and Poirot [19]. Janus produces 7.4×/5.9× more valid harnesses, and achieves 5.8×/6.5× the bitmap coverage compared with Atlas++ and Poirot, respectively. These results suggest that Janus can exercise more and deeper native code functionalities, exposing more potential security vulnerabilities and defects. Janus further uncovers 6 previously unknown memory-corruption vulnerabilities in closed-source native libraries shipped by mainstream Android vendors (e.g., Xiaomi, Vivo).

This paper makes the following three contributions:

- We propose Janus, the first agentic harness-synthesis approach toward JNI-aware fuzzing of Android native libraries, paving the way for broader auto-harnessing techniques across the Java-native boundary.
- We present algorithms to guide agentic workflows for automated harness generation that resolve explicit and implicit call dependencies and recursively construct composite arguments, backed by structure-aware retrieval, validation-guided refinement, and harness memory.
- Janus outperforms the state of the art in harness synthesis, execution validity, and code coverage, and further uncovers previously unknown vulnerabilities in closed-source native libraries shipped with mainstream Android vendor apps. For future research, we have released our prototype and artifacts on GitHub [27].

2 PRELIMINARIES

2.1 Java Native Interface in Android

In Android, the Java Native Interface (JNI) lets apps declare native methods with the `native` keyword and provide a matching C/C++ implementation. Methods are bound either statically through naming conventions resolved by `dlsym`

at load time, or dynamically via `JNI_OnLoad` calling `RegisterNatives` [5]. Once bound, each native call receives a `JNIEnv*` pointer, an opaque handle that mediates all interaction with the managed heap. Because native code cannot directly access Java object layouts, the JNI specification exposes a reflective API through `JNIEnv`: reading a field requires consecutive calls to `FindClass`, `GetFieldID`, and `Get<Type>Field`; object creation follows a similar sequence of `FindClass`, `GetMethodID` (for `<init>`), and `NewObject` [64]. For nested or user defined composite types, these reflective lookups should be chained layer by layer, as illustrated in Fig. 1 (lines N9-N16).

2.2 LLM Agents for JNI-Aware Harness Generation

LLM agents, including OpenCode [8], Claude Code [9], and OpenAI Codex [47], combine language-model reasoning with external tools to inspect program artifacts and execute generated code [24, 73, 75]. Operating in an iterative loop of code reasoning, planning, tool invocation, and self-reflection, the agent identifies missing information, queries relevant code, and revises its output based on execution observations [58, 71]. For JNI-aware harness generation, the required program context spans Java-side call sites and constructors as well as native-side implementations; by inspecting these artifacts, the agent investigates which invocations establish the required runtime state and how argument objects should be initialized, thereby connecting app-specific API usage knowledge across the language boundary.

Within a limited context window, the agent needs to acquire new cross-language code facts while retaining relevant intermediate findings [50]. To organize such multi-step tasks, OpenCode supports Skills that encode reusable procedural instructions and are loaded into the agent's context on demand [2]. These Skills specify task decomposition, intermediate outputs, decision criteria, and validation and termination conditions, while tools provide executable operations for code retrieval, compilation, and runtime testing [49]. Janus adopts OpenCode because its modifiable open-source implementation and support for custom tools and Skills facilitate integrating the JNI-aware code retriever with the algorithm-guided harness synthesis and refinement workflow, while also promoting research reproducibility.

2.3 Motivating Example and Challenges

We motivate Janus by exposing two challenges *inherent* to JNI-aware harness generation (§2.3.1, §2.3.2), using a concrete example (Fig. 1). §2.3.3 then shows, through a preliminary study, that naively delegating the task to an LLM agent fails to resolve these obstacles and instead introduces a third, *solution-induced* challenge.

Specifically, during normal execution of the real-world decompiled app in Fig. 1 (spanning decompiled Java lines J1–J18 and native lines N1–N16), `MyActivity.onCreate` loads `nativelib` and invokes `nativeInit` (J3) with a configuration string to initialize the internal library state (J12–J14, N3–N5); subsequently, `handleData` instantiates the nested user-defined classes `SubConf` (J9–J11) and `ProcOpts` (J5–J8) to invoke the target JNI method `nativeProcess` (J4, J15–J18), which verifies this state and reflectively retrieves nested fields from `opts` before reaching its core data-processing logic (N6–N16). However, synthesizing an effective fuzzing harness that achieves **valid execution** of a target JNI method proves to be challenging. Harnessing closed-source Android native libraries inherently presents difficulties because the prerequisites governing valid execution are intertwined across the Java and C/C++ boundary. Furthermore, meaningfully exercising the target JNI method not only requires recovering the cross-language runtime context, spanning prior native state initialization and Java-side setup (§2.3.1), but also demands synthesizing nested composite argument objects whose internal field requirements are implicitly dictated by the native implementation (§2.3.2). Below, we delve into each of these obstacles in detail.

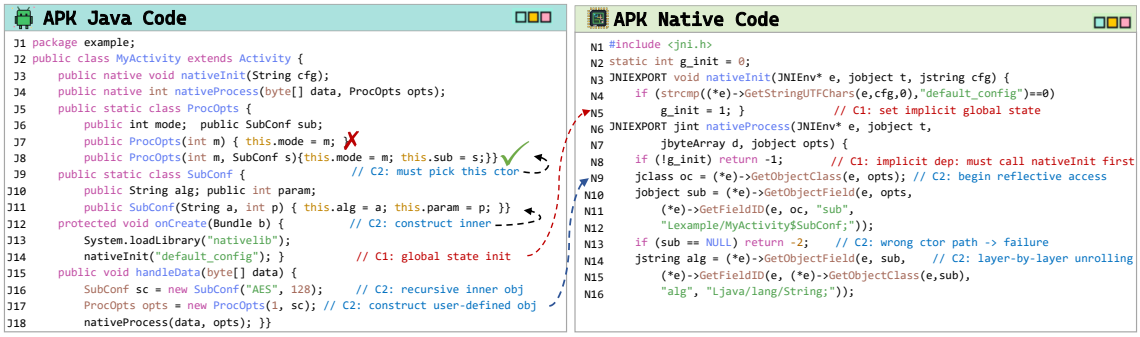


Fig. 1. Motivating case.

2.3.1 ***Hidden App-Specific Invocation Context.*** JNI method invocations typically depend on app-specific context established by preceding Java-side and native-side operations. Consider `nativeProcess` in Fig. 1: its valid execution depends on two pieces of runtime context. (*i*) The `ProcOpts` object passed from the Java side needs to be initialized through the constructor `ProcOpts(int, SubConf)` (J8) so that its nested field `sub` is non-null (J6). (*ii*) The native-side global flag `g_init` must have been set to 1 by the prior JNI call `nativeInit` (N5). If either condition is missing, `nativeProcess` returns an error code at its early guard checks (N8 or N13) and never reaches the core logic. Existing work attempts to recover such calling context through static analysis, yet with limited effectiveness. For instance, Poirot abandons native-side analysis entirely and extracts call sequences solely from the Java side: expanding outward from the callsite `handleData` of `nativeProcess`, it obtains only `nativeProcess` itself, while the critical `nativeInit` resides in a separate entry point `onCreate` that lies outside the call chain and is therefore omitted.

2.3.2 ***Missing Object-Construction Semantics.*** In real-world applications, JNI methods routinely accept complex objects (e.g., user-defined composite objects such as `ProcOpts` and its inner `SubConf` in Fig. 1, lines J4–J11) whose construction needs to be unrolled layer by layer across the language boundary. Because the fuzzing harness executes on the native side where Java object parameters are exposed as opaque references (e.g., `jobject opts` at line N7), the harness cannot directly populate object memory layouts; instead, it must assemble these Java objects from scratch via reflection-style JNI APIs (e.g., `FindClass`, `GetMethodID`, and `NewObject`). However, how a composite object should be constructed cannot be inferred from type signatures or Java class definitions in isolation, as it requires aligning Java-side instantiation paths with the fields internally accessed by the native implementation. Specifically, in Fig. 1, `nativeProcess` reads `ProcOpts.sub.alg` step by step through a chain of JNI reflective calls (lines N9–N16): it first reflectively retrieves `opts.sub` (aborting if `sub == NULL` at lines N10–N13) and subsequently reads `sub.alg` via `GetFieldID`/`GetObjectField` (lines N14–N16). Satisfying these native-side field accesses requires the harness not only to select the correct constructor `ProcOpts(int, SubConf)` (line J8) over `ProcOpts(int)` (line J7), but also to recursively construct the inner `SubConf` object via `SubConf(String, int)` (line J11) and supply a legal value for its `alg` string field. Existing methods [19, 66] are powerless in this regard, as both of them largely neglect to provide a means to synthesize complex Java objects, making it difficult to achieve a valid execution of a target JNI method and leaving JNI methods that consume composite types largely untestable.

2.3.3 ***LLM Agents Are Not a Silver Bullet.*** Given that overcoming both challenges demands reasoning over scattered cross-language code semantics rather than relying on rigid static heuristics, a natural question arises: *can modern LLM*

Prompt for the General-Purpose LLM Agent

Role: You are a software engineering expert responsible for generating fuzzing harnesses for JNI methods in closed-source Android apps.

Objective: Given the target app's APK, generate JNI-aware fuzzing harnesses that exercise its native methods using the Android Runtime (ART) in an Android emulator.

Input: APK file of the target application: `{target_apk}`.

Available Tools: JADX and Ghidra for inspecting decompiled Java and native code, together with an Android emulator for executing the generated harnesses.

Task Requirements:

1. Inspect the APK to identify its JNI methods and their corresponding native implementations.
2. Generate C/C++ harness code that loads the target native libraries and invokes the JNI methods with fuzzer-controlled inputs.
3. Provide the runtime initialization and argument preparation required to invoke the target methods in ART.
4. Attempt to build and execute the generated harnesses in the emulator. Use the observed errors to revise the code, and report any unresolved issues.

Expected Output: Return the harness source code and the instructions required to build and run it. Identify the target JNI methods and report whether the harnesses were successfully built and executed.

Fig. 2. Prompt template for the general-purpose LLM agent.

agents (§2.2) directly bridge this gap by autonomously inspecting APK artifacts and synthesizing JNI-aware harnesses? To answer this question, we conducted an exploratory study using a state-of-the-art general-purpose coding agent.

We randomly sampled 10 Android application packages (APKs), comprising 1,450 JNI methods in total, from the dataset of Poirot [19]. We instantiated the general-purpose LLM-based agent using Claude Code [9], with Claude Opus 4.5 [1], one of the most capable publicly available models leading major coding and agentic software engineering leaderboards (e.g., SWE-bench and LiveCodeBench) at the time of this study [17, 26, 29], as the underlying model. We provided the agent with basic reverse-engineering tools, including JADX [59] and Ghidra [46], as well as an Android emulator. We imposed no budget limit on this experiment. To keep the input conditions consistent, the agent received only APK files as input artifacts, the same input type as the full Janus implementation. Fig. 2 presents an expanded prompt template for this task.

Two authors cross-validated the generated results for all 1,450 JNI methods, confirming that the off-the-shelf agent generated 45 buildable and 13 runnable harnesses, which involved simple JNI functions with basic parameters. By inspecting its execution traces, we identified three compounding failure modes, which jointly indicate that the unconstrained reasoning of an off-the-shelf, general-purpose LLM agent is insufficient for the app-specific cross-language semantic reasoning demanded by §2.3.1 and §2.3.2.

(*i*) **Aimless exploration.** Decompiled real-world APKs can reach hundreds of thousands of lines, with code facts scattered across both the Java and C/C++ sides. At this scale, the agent either sees only local slices due to limited context capacity, or wanders through irrelevant files and conflates syntactically similar but semantically unrelated fragments, building its reasoning on fragmented and unverifiable snippets. (*ii*) **Inconsistency over long horizons.** Harnessing a single JNI method requires dozens of reasoning steps, and an app often contains hundreds of targets sharing parameter types and precursor dependencies. Without coordination, the agent repeatedly unfolds the same sub-problems, quickly exhausting the context window; as the dialog grows, attention to earlier facts decays, leading to self-contradictory call sequences or forgotten construction procedures. (*iii*) **Hallucination without validity refinement.** Even when equipped with basic compilation and execution tools, the agent fabricates non-existent entities (e.g., fictitious field

names, constructors, or JNI return-value semantics) and relies on them in subsequent reasoning. More subtly, it interprets "no crash" as success, while execution may complete only shallow steps without reaching the core logic, yielding no effective coverage and masking real defects.

3 Approach

To address the challenges of generating harnesses for Android native libraries, we introduce Janus, a tailored agentic approach for cross-language JNI-aware harness generation. Janus unfolds in three key stages, as shown in Fig. 3. Janus takes an APK file as input and employs a divide-and-conquer strategy, separating it into DEX bytecode (`.dex`) and native libraries (`.so`). Specifically, ❶ *Cross-Language Parser* grounds downstream harness synthesis by recovering a unified view of Java–native interactions, dispelling the opacity introduced by JNI indirection. ❷ *Harness Synthesizer*, built on a structure-aware retrieval layer, iteratively recovers each JNI method's precursor call sequence, uncovering both explicit and implicit dependencies, and recursively assembling its argument objects in a bottom-up manner; in parallel, the harness memory persists verified intermediate artifacts across methods, alleviating the context pressure of long-horizon reasoning. ❸ *Feedback-Driven Harness Refiner* constrains the generation space with app-specific harness templates and refinement, attributing compilation errors, runtime exceptions, and coverage signals back to the upstream stage responsible for repair, thereby suppressing the hallucinations and omissions that accumulate in long-chain reasoning.

3.1 Phase 1: Cross-Language Parser

Collectively, the cross-language analyses provide Janus with the initial cross-language context needed to trace how the consumer application invokes and propagates objects into the native library, paving the way for subsequent harness synthesis that mirrors realistic API usage patterns [75]. On the Java side, Janus extracts structural information for each defined class and method, such as fully qualified signatures, owning classes, argument and return types, and inter-method call relationships. Following Java-side parsing, Janus analyzes the native libraries and extracts function bodies along with the implicit constraints they impose on incoming arguments (e.g., validity checks on file descriptors or object fields).

To further bridge the two sides, Janus resolves the runtime address of every native method on-device through the ART, covering both static registration via `dlsym` and dynamic registration via `RegisterNatives` hooking [5]. The resulting *JNI binding table* records, per method, its fully qualified Java signature, argument and return types, owning `.so`, and relative offset. Cross-language analyses ground Janus's understanding in a unified view of Java–native interaction behavior, thereby laying the foundation for synthesizing harnesses that systematically exercise JNI functionality.

3.2 Phase 2: Harness Synthesizer

The Harness Synthesizer, the core stage of Janus, synthesizes JNI-aware harnesses designed to faithfully exercise app-specific Java-Native interactions, equipped with the cross-language decompiled artifacts (§3.1). This stage comprises four components. *Structure-Aware Code Retriever* equips Janus with cross-language code retrieval and comprehension capabilities. Building atop this, the *Dependency-Driven Call Sequence Extractor* addresses the question of *how a JNI method should be invoked*, producing JNI method cross-language call sequences; the *Recursive Argument Constructor* then operates on these sequences to resolve *how arguments should be constructed*, producing argument construction schemes. Spanning the entire stage, the *Harness Memory Manager* persists verified intermediate artifacts, enabling Janus to

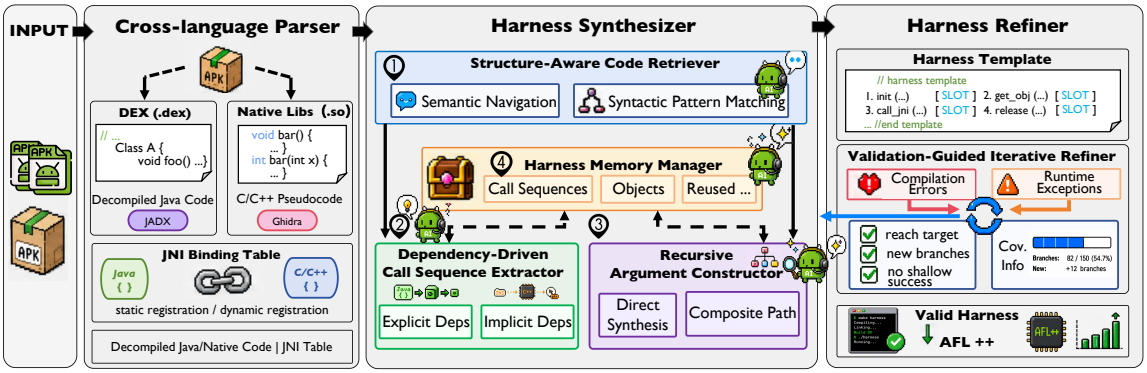


Fig. 3. Overview of JANUS.

reuse validated components across the JNI boundary, thereby mitigating redundant exploration and re-verification and alleviating the context pressure inherent in long-horizon tasks.

3.2.1 ***Structure-Aware Code Retriever.*** The decompiled artifacts from the Cross-Language Parser are voluminous, with harness-relevant facts scattered across decompiled Java code and native pseudocode. Textual retrieval alone struggles to reliably capture such cross-language associations, causing critical facts to be lost during the reasoning pass of LLM agents and thereby degrading the quality of the synthesized harness. To address this, JANUS builds a structure-aware retrieval layer atop the decompiled code, allowing the underlying agent to query along the symbol relationships and syntactic structures inherent in the code itself.

• **Semantic Navigation.** Existing off-the-shelf language servers (LSs), namely *jdtls* [20] for Java and *clangd* [41] for native code, implement the Language Server Protocol (LSP), but each is confined to a single language and cannot support bidirectional Java↔native navigation. To bridge this gap, JANUS implements a JNI-aware cross-language LS that augments the two underlying servers with the JNI method table (§3.1), thereby linking their language-specific symbol indexes. For a cross-boundary query, such as *Java declaration → native implementation* or the reverse, our server resolves the mapping, returning the target location together with its owning library and relative offset. Once the query reaches either side, the cross-language LS delegates intra-language reference, definition, and type queries to the corresponding underlying LS. This design turns the two otherwise disjoint symbol indexes into a unified, traversable cross-language symbol table.

• **Syntactic Pattern Matching.** Resolving JNI behavior requires identifying structural code patterns, such as native functions that read a receiver field via `GetFieldID`, which cannot be expressed as symbol queries. JANUS therefore defines two sets of AST-Grep templates: Java-side templates capture declarations of native functions and object-propagation paths (e.g., constructors, setters, field read/writes), while native-side templates capture JNI field and object accesses (e.g., `GetFieldID`). All matches are returned uniformly as ⟨*location*, *symbol info*, *code snippet*⟩ triples.

JANUS adaptively selects among these tools as the LLM agent's reasoning proceeds, so that each synthesis decision is traceable to a specific source location, equipping the subsequent phase with the unified cross-language view required to generate far-reaching JNI-aware harnesses.

3.2.2 ***Dependency-Driven Call Sequence Extractor.*** JANUS recovers the app-specific runtime context of prerequisite calls for each target JNI method, enabling the harness to exercise execution paths legitimately reachable

by the consumer application [19]. However, due to the closed-source nature of Android native libraries, JNI usage knowledge is often sparse (or even missing), making app-specific runtime context recovery inherently challenging: **incomplete or misordered prerequisite calls may silently corrupt the execution environment, inducing spurious crashes** [66]. The key, therefore, is for the harness to faithfully reconstruct the prerequisite context on which the target JNI method depends during legitimate execution in a real-world application. By examining the execution constraints of target JNI methods in real apps, we identify two categories of dependency that should be satisfied:

• **Explicit Dependencies.** Constraints that can be established entirely on the Java side. Concretely, an argument object needs to be instantiated through a Java constructor, and any of its fields that the native code will subsequently read must have been explicitly assigned by Java code. Satisfying such constraints involves only Java-side object construction and field writes, without crossing the JNI boundary.

• **Implicit Dependencies.** Constraints that should be established on the native side by preceding JNI calls. They manifest as fields on an argument object written back from native code (e.g., C pointers stored as `long`), global context maintained inside a native library, or argument values consumed by a later call but produced as the return value of an earlier one. Their common feature is that the relevant values or state cannot be readily fabricated on the Java side and must be produced by some preceding JNI call during native execution [66].

The dependencies are app-specific and intertwined across the Java and native sides, making them challenging to acquire for reconstructing the invocation context, where static analysis faces an inherent precision-scalability trade-off. Janus therefore casts this recovery as a dependency-driven algorithm that treats unmet dependencies as the driving force, iteratively resolving both explicit and implicit dependencies needed to reach the target JNI method. Within this algorithm, Janus replaces handcrafted heuristics with LLM agents that capture diverse app-specific conventions, identifying missing dependencies and the calls that satisfy them.

We formalize the above solving process as follows. Given a target JNI method m, its prerequisite call sequence is denoted $S_m = \langle c_1, c_2, \ldots, c_k, m \rangle$, where each c_i is a Java method or JNI method that contributes to establishing the execution context of m. Every prerequisite call in the sequence is paired with a structured annotation $a_i = \langle c_i, role_i, source_i, binding_i \rangle$: $role_i$ identifies the dependency category that c_i satisfies, $source_i$ records the JNI-related code supporting its inclusion, and $binding_i$ describes how the products of c_i are consumed downstream: *(i)* how its return value flows into the argument of a subsequent call and *(ii)* how its side effects (e.g., writes to object fields, initialization of global context) establish a particular dependency state for later calls or for the target JNI itself. The set of currently unsatisfied dependencies of m is denoted $D_m = \{d_1, d_2, \ldots, d_n\}$, where each d_j corresponds to an unsatisfied instance of one of the dependency categories introduced above. The objective is to construct S_m together with its annotation set A_m such that $D_m \subseteq \text{Resolution}(S_m, A_m)$, where $\text{Resolution}(S_m, A_m)$ denotes the set of dependencies that the current sequence and its annotations are able to establish. The recovered invocation context is considered *resolved* once currently identified dependencies D_m are satisfied.

Janus guides this resolution process through an agentic iterative loop (Algorithm 1). The input consists of the target JNI method m and the *Code Retriever* toolset $\mathcal{T}$; the output is an accepted prerequisite call sequence S_m paired with its structured annotations A_m, or $fail$ if no such sequence can be assembled within the search budget. Specifically, S_m is first initialized to contain only the target JNI method itself, A_m is initialized to empty, and the initial dependency set D_m is extracted from m's Java declaration and native implementation (line 1). The sequence and its annotations are then gradually augmented as D_m is progressively resolved during the loop.

In each round, Janus generates targeted queries from the currently unsatisfied dependencies (line 3), invokes $\mathcal{T}$ to retrieve JNI-related code (line 4), turns the retrieved results into candidate calls (line 5), and inserts them into the current

Algorithm 1: Call Sequence Extraction

Input: m: target JNI method; Γ: decompiled code; $\mathcal{T}$: tool set
Output: S_m: accepted prerequisite call sequence; A_m: annotation set, or FAIL

1 $S_m \leftarrow \langle m \rangle$; $A_m \leftarrow \emptyset$ $D_m \leftarrow$ InitialDependencies$(m, \Gamma, \mathcal{T})$
2 **while** $D_m \neq \emptyset$ ***and*** *budget remains* **do**
3 $\quad Q \leftarrow$ Agent.GenerateQueries(m, S_m, D_m)
4 $\quad E \leftarrow$ ExecuteTools$(Q, \mathcal{T})$
5 $\quad C \leftarrow$ Agent.ProposeCalls(E, D_m)
6 $\quad \langle S_m, A_m \rangle \leftarrow$ InsertAndAnnotate(S_m, A_m, C, E)
7 $\quad result \leftarrow$ Agent.CheckResolution(m, S_m, A_m, D_m)
8 $\quad$ **if** $result ==$ *RESOLVED* **then**
9 $\quad\quad D_m \leftarrow \emptyset$;
10 $\quad$ **else**
11 $\quad\quad D_m \leftarrow result$.missing_dependencies
12 **if** $D_m \neq \emptyset$ **then return** FAIL
13 $verdict \leftarrow$ Agent.GlobalReview(m, S_m, A_m)
14 **if** $verdict ==$ ACCEPT **then**
15 $\quad$ **return** S_m, A_m
16 **else return** FAIL

sequence together with their structured annotations (i.e., *role*, *source*, *binding*, line 6). After insertion, JANUS switches to an auditor's perspective and checks whether the sequence has been locally resolved, with the remaining unsatisfied dependencies (or $\emptyset$ upon resolution) carried into the next round ((lines 7–11). If the search budget is exhausted before closure, the algorithm returns failure (line 12). At the start of each round JANUS first consults the memory manager (§3.2.4): if a previously recorded sequence already covers a current dependency, it is reused directly to avoid redundant solving. Once D_m is satisfied, JANUS performs a global review over the entire sequence (line 13), checking that the call ordering, dependency consistency, and binding relations indeed hold at the sequence level; only sequences that pass this review are finally accepted (lines 14–15).

Through this loop, JANUS reframes call sequence construction into a code-grounded resolution process driven by missing dependencies, with the global review checking sequence-level consistency. The output is the set of calls (together with their supporting code) required to reconstruct the target JNI's execution context, which serves as the input to the subsequent argument synthesis stage.

3.2.3 ***Recursive Argument Constructor.*** Producing executable harnesses further requires assembling argument objects from scratch via reflection-style JNI functions [34], a non-trivial task since JNI methods commonly accept complex composite types whose construction logic cannot be inferred from type signatures alone: **failing to align Java-side instantiation paths with the fields internally accessed by the native implementation leaves inner objects unpopulated, causing the native method to abort without achieving a valid execution**.

To tackle this challenge, JANUS recursively unrolls the construction of composite objects layer by layer, selecting appropriate Java constructors and populating nested fields to satisfy native-side accesses. Specifically, Algorithm 2 takes as input the target argument type τ, the call sequence S_m and its annotations A_m produced in §3.2.2, and the tool set $\mathcal{T}$ from §3.2.1; it returns either the construction code $ctor(\tau)$, which can be embedded into the harness, or *fail* when no feasible candidate path exists.

Algorithm 2: Recursive Argument Object Construction

Input: τ: target argument type; S_m, A_m: call sequence and annotations (§3.2.2);
Γ: decompiled code; $\mathcal{T}$: tool set
Output: $ctor(\tau)$: construction code that produces an argument of type τ, or FAIL

// **Direct argument resolution (binding, synthesis, and reuse)**
1 **if** $Bound(\tau, A_m)$ **then return** $EmitBoundValue(\tau, A_m)$
2 **if** $Synthesizable(\tau)$ **then return** $EmitJNISynthesis(\tau)$
3 **if** $Memory.Hit(\tau, S_m, A_m)$ **then return** $Memory.Get(\tau, S_m, A_m)$
// **Composite object path selection**
4 $ctorCands \leftarrow$ QueryConstructorsAndFactories$(\tau, \Gamma, \mathcal{T})$
5 **while** $ctorCands \neq \emptyset$ **do**
6 $\quad \pi \leftarrow$ Agent.SelectConstructionPath$(ctorCands, S_m, A_m, \Gamma)$
7 $\quad ctorCands \leftarrow ctorCands \setminus \{\pi\}$; $argCodes \leftarrow \emptyset$
$\quad$ // **Sub-argument recursion and assembly**
8 $\quad$ **foreach** $\tau' \in ParamTypes(\pi)$ **do**
9 $\quad\quad ctor(\tau') \leftarrow$ RecConstruct$(\tau', S_m, A_m, \Gamma, \mathcal{T})$
10 $\quad\quad$ **if** $ctor(\tau') =$ FAIL **then break to next candidate in Line 5**
11 $\quad\quad argCodes$.add$(ctor(\tau'))$
12 $\quad ctor(\tau) \leftarrow$ EmitNewObject$(\pi, argCodes)$
$\quad$ // **Local Check and memoization**
13 $\quad$ **if** $LocalCheck(ctor(\tau))$ **then**
14 $\quad\quad$ Memory.Put$(\tau, S_m, A_m, ctor(\tau))$
15 $\quad\quad$ **return** $ctor(\tau)$
16 **return** FAIL

• **Direct Argument Resolution.** This step defines the termination condition (i.e., leaf case) of the recursion: before attempting any construction path, JANUS first checks whether the target type τ can be produced directly, terminating the recursion immediately if any of three conditions holds (lines 1–3). (*i*) τ is already bound in A_m (e.g., a fixed setup literal such as `"default_config"` in Fig. 1 or a native handle returned by an earlier call in S_m), in which case JANUS directly emits the bound constant or variable reference. (*ii*) τ is an unbound leaf type that is *directly synthesizable*, i.e., it requires no nested object assembly. This covers ① primitive types, obtained by slicing the corresponding number of bytes from the fuzzer stream and decoding them (e.g., taking 4 bytes and decoding them as a `jint`), and ② reference types such as `String` or primitive arrays, parsed from bytes and materialized as Java objects via the corresponding JNI API (e.g., `NewStringUTF`). (*iii*) A compatible construction for τ that covers the fields and bindings required by (S_m, A_m) has already been recorded in the harnessing memory (§3.2.4), in which case JANUS reuses it directly, short-circuiting the recursion to eliminate redundant reasoning.

• **Composite Object Path Selection.** When τ cannot be synthesized directly, JANUS enumerates candidate construction paths *ctorCands*, including Java constructors, factory methods, and instantiation sites of τ observed in the app (line 4), based on the structure-aware retrieval tools $\mathcal{T}$. This step is difficult for purely heuristic filling: a given type often exposes multiple constructors or factory methods that differ in their initialization scope, side effects, and prerequisite dependencies, and these differences are hard to tell apart from type signatures alone. JANUS therefore guides the LLM agent's decision-making via explicit task specifications that weigh two factors: (*i*) how the type is instantiated in the app, and (*ii*) how well each candidate covers the fields read on the native side, to select the path (π) that matches the object state expected by the native code (line 6). For example, for `ProcOpts` in Fig. 1, JANUS selects `ProcOpts(int, SubConf)` (line J8), because this constructor initializes the `sub` field that the native side later reads (lines N10–N13).

• **Sub-Argument Recursion and Assembly.** Once a path π is selected, JANUS recurses on each parameter type $\tau' \in \text{ParamTypes}(\pi)$ to obtain the corresponding sub-argument construction code $ctor(\tau')$ (lines 8–9); if any sub-argument cannot be constructed, JANUS discards π, backtracks to the path-selection loop to try a remaining candidate from *ctorCands* (lines 5–10), and returns *fail* once *ctorCands* is exhausted (line 16). After all sub-arguments are obtained, JANUS emits the reflective JNI API calls (e.g., `FindClass`, `GetMethodID`, and `NewObject`) that assemble the object at the current level (lines 7–12). Consider `ProcOpts` (lines J5–J8 in Fig. 1): after selecting π = `ProcOpts(int, SubConf)` (line J8) over `ProcOpts(int)` (line J7), JANUS recurses on its two formal parameter types, `int` and `SubConf`. While `int` terminates immediately as a directly synthesizable leaf type, the recursive step on the composite type `SubConf` (lines J9–J11) selects `SubConf(String, int)` (line J11) and further recurses on its two leaf parameter types {`String`, `int`}, forming the Java-side expansion chain `ProcOpts` → `ProcOpts(int, SubConf)` → {`int`, `SubConf`} → `SubConf(String, int)` → {`String`, `int`}. This expansion chain is induced on the fly by the recursive calls over parameter types and tracked via the call stack and the agent's context, whereas only verified constructions are persisted in the harness memory(§3.2.4) for subsequent reuse.

• **Local Check and Memoization.** Once assembly is complete, JANUS performs a local check on $ctor(\tau)$. First, it verifies that all referenced classes, constructors, fields, and JNI method signatures exist in the code artifacts. Second, it runs a smoke test to check whether the object can be instantiated, fields can be written, and the construction does not raise a Java exception (line 13; otherwise, it backtracks to try the next candidate path). Only $ctor(\tau)$ that passes this check is added to the Memory in §3.2.4 for reuse by subsequent JNI methods (lines 14–15).

Overall, JANUS produces, for every argument type along the call sequence, a piece of construction code grounded in either bound context values or the fuzzer byte stream. Together with the call sequence from §3.2.2, these snippets form the invocation context for the target JNI method.

3.2.4 ***Harness Memory Manager.*** While §3.2.2 and §3.2.3 can synthesize call sequences and argument constructions for individual JNI methods, processing hundreds of JNI targets within the same app introduces substantial redundancy. Specifically, the same composite argument types and precursor initialization routines frequently recur across related JNI methods; deriving them from scratch for each target repeatedly invokes retrieval tools, regenerates boilerplate code, and re-runs smoke tests. Because harness synthesis is inherently a long-horizon task, such repetition not only inflates LLM reasoning and verification overhead, but also fills the conversation history with redundant traces, making it essential to manage the working memory of the LLM agent to mitigate the constraints of its limited context window and the degradation of long-range consistency.

To this end, JANUS introduces a *Harness Memory Manager* that maintains a global store of verified harness components. Whenever an intermediate component is validated (e.g., a call sequence that passes the resolution check, or an argument construction plan that passes the smoke test), the *manager* commits it to the memory alongside the metadata of the dependencies or object fields it initializes. In subsequent rounds, when solving a new requirement, the *manager* first queries the memory and reuses a matching component directly if a compatible one is already available, namely one whose established state or initialized fields cover the requirements in (S_m, A_m). If a reused component later fails downstream harness validation (§3.3.2) due to stricter target-specific state constraints, the *manager* retains the entry in global memory for other methods with simpler requirements, while bypassing it for the current target so that an alternative path can be synthesized.

3.3 Phase 3: Feedback-driven Harness Refiner

With prerequisite call sequences and argument construction plans synthesized, Janus assembles them into runnable harnesses for fuzzers such as AFL++ [34], but two challenges arise. First, the JNI-aware harness should satisfy a chain of strict constraints (e.g., library loading, `JNIEnv` acquisition), and letting an LLM emit the entire harness in one shot opens a generation space wide enough for hallucinated, malformed code to slip through undetected. Second, harness validity cannot be judged by "does not crash" alone [25]: a harness that runs to completion may never enter the target JNI method's core logic, yet LLM agents readily mark such executions as successful, leaving downstream fuzzing trapped on shallow paths. Janus addresses these challenges with two designs.

3.3.1 ***App-Specific Harness Template.*** Janus constrains the LLM agent's generation space with a standardized JNI harness template that defines the overarching execution skeleton and the precise topological order in which individual functional sections are wired together, as illustrated in Fig. 3; rather than synthesizing the entire harness from scratch in an unconstrained, error-prone manner, the agent is guided to populate designated slots with content derived from the previous two phases.

3.3.2 ***Validation-Guided Iterative Refinement.*** Janus closes the synthesis loop through feedback-driven repair and serves as an empirical validation boundary [56] for the preceding phases. After assembling a harness, Janus validates it with three signals [56]: *(i)* compiler diagnostics, *(ii)* runtime exceptions, and *(iii)* coverage feedback, constraining the LLM agent to perform targeted refinement rather than unconstrained regeneration. Specifically, using the target method's resolved library offset (§3.1) and Frida-based edge instrumentation, Janus executes the harness against a set of mutated trial inputs and considers it valid only if execution hits the target entry address and progressively covers new branches beyond the function prologue. This criterion overrides the agent's naive "no-crash" self-assessment and prevents shallow executions that repeatedly exit at initial guard checks (e.g., lines N8 and N13 in Fig. 1) from being misjudged as successful.

When validation fails, Janus implements failure attribution to identify the likely source of each failure, guiding the *Harness Refiner*: failures caused by malformed harness code, such as compilation or linking errors, are handled in §3.3 by re-synthesizing the affected template slots, whereas failures indicating invalid argument state or missing prerequisites, such as null-pointer dereferences, type mismatches, uninitialized native handles, or stagnant branch coverage at entry checks, are routed back to §3.2, which bypasses any previously reused memory construction for the current target and revises the corresponding argument-construction plan or prerequisite call sequence. Through this iterative loop, §3.3 suppresses hallucinations and omissions accumulated across long-horizon reasoning, ensuring that every harness delivered to the fuzzer is grounded in dependencies and object states that have been verified on-device.

4 Implementation

As shown in Fig. 4, Janus implements the proposed algorithms through task-specific Skills that guide the agent's cross-language information gathering, synthesis decisions, and validation. Specifically, these Skills encode the agentic workflow as procedural instructions within a customized extension of OpenCode [8]. Below, we describe four key aspects of how the agent follows these instructions.

• **Task Orchestration.** Janus uses OpenCode agent to orchestrate the Cross-Language Parser (§3.1), the Harness Synthesizer (§3.2), and the Harness Refiner (§3.3). The project-level orchestration instructions in `src/AGENTS.md` [27] specify the execution order of these stages and how refinement requests are routed. At each stage, the agent loads

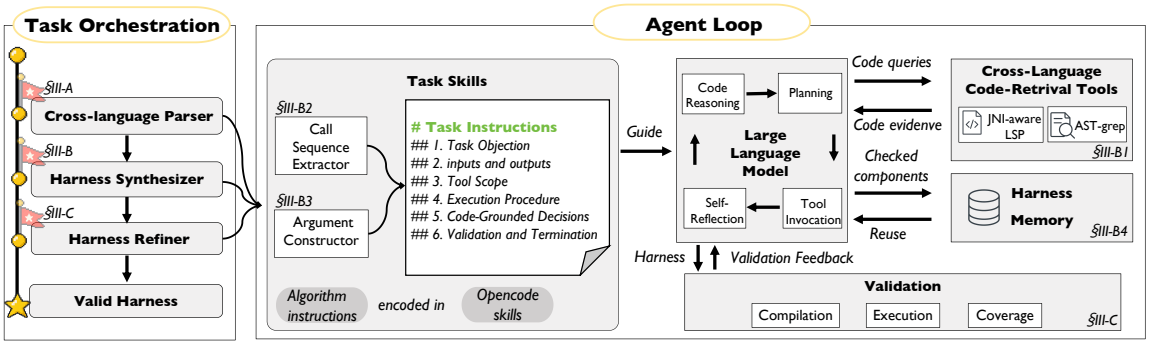


Fig. 4. Agentic implementation of Janus.

the corresponding Skill from `src/.opencode/`[27]. These Skills specify the overall workflow as concrete procedures, including JNI binding resolution, dependency-driven call sequence extraction, recursive argument construction, and continuous validation.

• **Algorithms Encoded as Skills.** Janus's algorithms (described in §3.2.2 and §3.2.3) explicitly specify task objectives, inputs and outputs, available tools, execution steps, decision criteria, and validation and termination conditions. The decision criteria distinguish explicit from implicit dependencies and directly synthesizable arguments from those requiring recursive construction. We encode our two key algorithms as OpenCode Skills to guide the agent's JNI-aware, app-specific execution loop. For Algorithm 1, the Skill directs the agent to formulate queries based on unresolved explicit and implicit dependencies, identify prerequisite calls, extend the call sequence, and reassess the remaining dependencies. Once these dependencies are resolved, the agent performs a global review of the sequence. For Algorithm 2, the Skill guides the agent through direct synthesis or memory reuse, construction-path selection, recursive sub-argument construction, and bottom-up assembly, with backtracking when a candidate construction fails. These instructions guide what the agent investigates, how it decomposes a synthesis task, and when it should continue, backtrack, or terminate.

• **Code-Grounded Agent Loop.** Within the algorithmic procedure encoded in each Skill, the agent combines code reasoning, planning, tool invocation, and self-reflection on intermediate results to recover app-specific JNI usage knowledge. This procedure involves identifying prerequisite method calls and determining how to construct complex Java arguments that satisfy implicit native-side expectations. Guided by multi-step planning, the agent retrieves relevant code through the JNI-aware LSP and AST-Grep interfaces (§3.2.1), which together provide the tool set $\mathcal{T}$ used by Algorithms 1 and 2. The agent then applies code reasoning over the retrieved code facts for dependency classification and construction-path selection, while leveraging self-reflection to validate intermediate hypotheses and prune infeasible paths at the corresponding algorithmic steps.

• **Checks, Memory, and Validation Feedback.** Validation checks determine whether synthesized components are accepted for reuse: call sequences undergo global review, while argument-construction code undergoes local checks, including smoke tests. Accepted components are stored in the Harness Memory Manager (§3.2.4) for subsequent reuse. Compiler diagnostics, runtime exceptions, and coverage measurements then provide feedback on the assembled harness. The failure-attribution mechanism (§3.3) uses this feedback to identify the likely source of failure and route refinement to the corresponding procedure. The agent revisits the relevant Skill to revise prerequisite calls, argument-construction code, or the affected harness template slots, connecting algorithm-guided synthesis with execution-based refinement.

5 Effectiveness Evaluation

Our evaluation of Janus addresses the following research questions (RQs).

- **RQ1**: How effective is Janus in generating JNI-aware fuzzing harnesses for closed-source Android apps?
- **RQ2**: How does each component of Janus contribute to harness generation?
- **RQ3**: Can Janus discover previously unknown bugs in real-world Android apps?

Experimental Environment. We conduct all experiments on a Mac Studio (M3 Ultra, 512 GB RAM). Janus exposes the structure-aware code retriever (§3.2.1) to the agent as tools under OpenCode's tool-calling convention. We adopt GLM-5 as the primary model throughout due to its cost-effectiveness [72]. Tests run on Android Emulator instances (API 31, 4 CPU cores, 6 GB RAM each), using Quarkslab's open-source Android runtime emulation scheme and Frida-driven AFL++ as the fuzzing engine [34].

Dataset. We evaluate Janus on two complementary datasets that target different aspects of its effectiveness.
• **Poirot-44.** We start from the public dataset released with Poirot [19], which contains 103 popular real-world apps collected from Google Play. To ensure reproducibility and a fair comparison with the state-of-the-art baseline, we re-run Poirot's officially released raw Docker image without modification on this dataset; the image successfully builds harnesses for only 44 of the 103 apps. We adopt these 44 apps, including 16 high-impact apps with over 500M downloads (e.g., Google and YouTube Music), as our first dataset, comprising 21,386 JNI methods in total. Poirot-44 is used in **RQ1** and **RQ2** to evaluate harness-generation effectiveness and the contribution of individual components, respectively.
• **Vendor-20.** To further assess Janus's ability to discover unknown vulnerabilities in up-to-date, security-sensitive apps, we collect a second dataset (**Vendor-20**) by extracting 20 pre-installed apps from the latest flagship devices of 4 mainstream Android vendors (Xiaomi, Vivo, OnePlus, and OPPO; three to five per vendor; all running the February 5, 2026 security patch [7]). All apps in Vendor-20 share three properties: *(i)* they are vendor-preinstalled and span diverse functionalities including image processing, audio decoding, and font parsing; *(ii)* they rely heavily on native libraries for complex, error-prone low-level operations; and *(iii)* they typically run with `system` privileges, so any native-side memory vulnerability they harbor is significantly more impactful once exploited. Vendor-20 is used exclusively in **RQ3** for in-the-wild vulnerability discovery.

5.1 RQ1: Effectiveness of Janus's Harness Generation

5.1.1 ***Setup***. We evaluate the effectiveness of Janus's harness generation on the Poirot-44 dataset, comparing against the two most relevant Android JNI fuzzing systems, Atlas++ [10] and Poirot. Note that the original Atlas [66] is not publicly available; we therefore compare against Atlas++ [10], an open-source, improved reimplementation released in April 2026. Of the 21,386 JNI methods extracted from Poirot-44, 13,933 are excluded before harness generation, leaving 7,453 JNI targets for which Janus attempts harness generation. Specifically, we exclude JNI methods for which no call sites are identified in the host app's DEX bytecode, following established practice [19].

To ensure a fair comparison, all three tools are driven by the same evaluation pipeline: identical APKs, emulator configurations, AFL++ instrumentation and seed corpora, and the same per-harness fuzzing budget. During the harness-generation phase, each tool is first allotted a 30-minute timeout for its static-analysis stage, and is then given an additional 30-minute timeout per JNI target method for harness construction. Following the Poirot evaluation protocol [19], each successfully built harness is allocated an AFL++ validation window of up to 4 minutes (approximately 29.6 days of

Table 1. Effectiveness comparison on Poirot-44 (best in **bold**).

	Janus				Atlas++				Poirot			
App	#Build	#Run	Cov	#Crashes	#Build	#Run	Cov	#Crashes	#Build	#Run	Cov	#Crashes
UCMobile	**578**	**505**	**22.6**	**2,920**	93	4	4.3	2	84	84	4.8	33
amazon.mShop	**394**	**383**	**23.1**	**293**	111	90	7.3	0	86	86	4.1	279
chrome	**613**	**582**	**43.2**	**262**	80	18	18.4	0	67	67	12.4	250
apusapps	**22**	**21**	**3.3**	0	8	2	0.4	0	4	4	1.1	0
billiards.pool	**26**	**21**	**7.7**	**4**	**26**	5	2.3	0	7	6	2.6	0
bitstrips	**86**	**86**	**8.9**	**130**	5	0	0.0	0	32	32	2.4	124
cleanteam	**23**	**15**	**21.2**	0	1	0	0.0	0	3	1	0.8	0
skirt	**36**	**33**	**5.4**	**15**	25	5	2.3	0	10	10	2.9	0
larkplayer	**19**	**19**	**11.1**	0	10	8	2.9	0	5	5	1.7	0
evernote	**20**	**19**	**2.0**	**8**	8	3	1.5	7	5	2	1.3	0
gokids	**19**	0	0.0	0	9	0	0.0	0	4	0	0.0	0
g.meetings	**21**	**20**	**10.2**	**18**	5	4	3.0	0	4	4	2.2	17
g.photos	**156**	**110**	**33.6**	**67**	20	10	19.1	0	21	21	8.7	64
yt.music	**54**	**50**	**28.7**	**45**	2	0	0.0	0	15	15	3.6	43
g.dialer	**12**	**12**	**24.0**	**25**	2	1	9.6	0	2	2	5.4	0
g.search	**213**	**189**	**19.1**	0	2	0	0.0	0	1	0	0.0	0
g.videos	**73**	**48**	**11.7**	**5**	29	4	2.5	0	13	9	3.2	4
hicloud.clone	**27**	**27**	**31.4**	**4**	17	17	3.2	3	8	8	2.1	0
huawei.maps	**160**	**160**	**13.7**	**29**	59	44	2.8	28	39	39	2.4	0
imdb	**146**	**108**	**10.7**	**10**	41	8	1.9	0	23	14	3.1	9
imo	**128**	**128**	**12.2**	**15**	66	52	2.2	14	33	33	1.8	0
supervpn	**4**	**4**	**15.6**	0	2	0	0.0	0	1	1	0.9	0
parallel	**31**	**28**	**4.6**	**3**	13	4	0.5	0	5	5	1.2	0
linkedin	**166**	**166**	**25.8**	**8**	46	35	2.2	7	23	23	1.6	0
skydrive	**2,462**	**2,410**	**26.5**	**225**	743	323	2.0	214	396	396	3.4	0
brain.out	**90**	**90**	**20.3**	**47**	32	31	2.1	45	27	27	1.9	0
talkinghank	**14**	**14**	**5.9**	**12**	4	2	0.7	0	2	2	0.6	0
tom	**31**	**15**	**0.6**	**14**	13	2	0.0	0	4	2	0.3	0
shealth	**36**	**36**	**8.2**	0	25	11	2.2	0	12	12	2.5	0
easyMover	**281**	**281**	**27.1**	**13**	72	71	2.7	12	40	40	2.3	0
printservice	**40**	**36**	**20.9**	**19**	2	0	0.0	0	1	0	0.0	0
skype	**715**	**14**	**11.8**	0	353	3	2.0	0	213	6	2.8	0
skylight	**67**	**56**	**69.7**	**57**	25	18	2.8	54	13	13	2.1	0
vectorunit	16	**14**	**2.7**	0	**17**	0	0.0	0	7	1	0.7	0
midrop	**24**	**19**	**38.1**	**40**	7	3	5.3	0	4	4	5.6	0
videoeditor	**87**	**26**	**7.3**	**7**	36	1	2.8	6	5	5	3.0	2
scaryteacher	**36**	**35**	**18.3**	**74**	8	4	1.8	0	2	2	1.4	0
zeropage	**58**	**58**	**1.8**	**860**	8	4	0.9	0	3	3	0.7	0
superbros	**23**	**16**	**1.2**	**2**	9	1	0.7	0	4	4	0.8	1
turbovpn	**54**	**49**	**12.2**	**15**	12	5	2.4	2	7	7	2.7	14
bubble.pop	**41**	**41**	**36.4**	**25**	25	4	2.3	0	16	16	4.2	24
amazon.in	**81**	**81**	**16.8**	**43**	20	16	2.8	10	18	18	3.1	41
pesam	**34**	**33**	**7.6**	**57**	22	4	3.0	0	4	4	2.4	0
firefox	**40**	**36**	**20.9**	**19**	6	2	2.8	0	1	1	1.5	0
Total	7,257	6,094	16.9	5,390	2,119	819	2.9	404	1,274	1,034	2.6	905

Note: Columns: **#Build**: harnesses that successfully compile, link, and package (no guarantee of valid execution). **#Run**: harnesses that execute as valid fuzzing targets, i.e., initialize the runtime, load the target native library, consume inputs by executing the target function, and produce growing coverage. **Cov (%)**: AFL++ bitmap coverage. Cov is measured only over the #Run harnesses. **#Crashes**: unique crashes from AFL's deduplication pipeline. The last row gives column totals (mean across apps for Cov). Janus's improvements are statistically significant (paired Wilcoxon, $p < 0.05$).

compute time per run across 10,650 buildable harnesses). To reduce the non-determinism of the LLM, we fix the model temperature to 0 and average the results of Janus over three independent runs per JNI target method.

5.1.2 ***Result.*** The comparison results are shown in Table 1. Janus consistently outperforms both baselines across all metrics. Among the 7,453 attempted JNI targets, Janus constructs 7,257 buildable harnesses (97.4%) and successfully executes 6,094 of them (84.0% of the built harnesses). These counts correspond to 3.4×/5.7× more buildable harnesses and 7.4×/5.9× more runnable harnesses than Atlas++/Poirot, respectively. In terms of fuzzing effectiveness, Janus achieves a per-app mean bitmap coverage of 16.9% (vs. 2.9% for Atlas++ and 2.6% for Poirot). After deduplication,

Janus retains 5,390 crashes across 35 apps, compared to 404 (13 apps) and 905 (14 apps) for the two baselines. These gains are primarily attributable to Janus's ability to capture complex, app-specific cross-language dependencies and complex construction knowledge, enabling it to generate more semantically meaningful fuzzing harnesses.

• **Failure Analysis.** Among the 7,257 harnesses generated by Janus, 6,094 satisfy the #Run criterion (84.0%); the remaining 1,163 fail to execute successfully, indicating that the *Harness Refiner* alone is insufficient to guarantee that all harnesses are repairable. To understand the root causes, we conduct a detailed manual analysis of these failures. When the iteration budget is extended to two hours per harness, 637 of the 1,163 failed harnesses eventually succeed, reflecting a cost–effectiveness trade-off rather than a fundamental capability ceiling. The remaining failures fall into three categories. *(i)* Feedback in the emulated environment is limited to coarse crash signals (e.g., SIGSEGV and its faulting address), from which the LLM cannot always derive correct repair guidance. *(ii)* Some target libraries depend on external runtime state, such as device sensors or system services, that cannot be satisfied within the current emulated environment, and such prerequisites cannot be eliminated through code-level iterative repair alone. *(iii)* Some target libraries contain anti-emulation or integrity checks that trigger early exits in an isolated harness environment, preventing stable execution regardless of harness-level repairs.

• **Crash Validation.** Since the analyzed libraries and apps are predominantly closed-source, manual analysis is highly labor-intensive. We select the 200 crashes with the highest coverage for manual inspection by three security researchers with over 3 years of experience (2–18 hours per crash), following a rigorous crash analysis procedure [19]. We first exclude 46 crashes that do not accept attacker-controlled inputs, i.e., whose crash paths cannot be reached by external inputs. Of the remaining 154 crashes, we further discard 65 false positives, i.e., crashes not caused by defects in the target function itself. Their root causes are threefold: (*i*) incorrect invocation patterns inferred by the LLM (12 crashes across 4 apps), (*ii*) erroneous code information produced by reverse-engineering tools due to obfuscation (23 crashes across 11 apps), and (*iii*) benign anomalies, including exceptions actively thrown by built-in exception-handling logic within the target method rather than genuine defects (30 crashes across 9 apps). Among the 89 retained crashes, we confirm the presence of defects in the corresponding functions. However, given that the tested app versions are dated (2022), the underlying vulnerabilities may have already been reported by others; we verify this by checking the latest versions of the respective libraries and confirm that all identified defects have since been patched.

• **Cost Analysis.** Averaged over three runs, each complete run of Janus consumes 2.57 billion tokens at an input-to-output ratio of 11.05:1, with a context-cache hit rate of 94.7%. The total API cost per run is $1,292.6, i.e., ~$0.212 per runnable harness, demonstrating the economic practicality of Janus.

Answer to RQ1: Janus effectively generates fuzzing harnesses for native libraries of Android apps. Over state-of-the-art baselines, Janus yields 3.4×/5.7× more buildable and 7.4×/5.9× more runnable harnesses, and uncovers 5,390 crashes.

5.2 RQ2: Ablation Study of the Analysis Passes

5.2.1 ***Setup.*** RQ2 quantifies the contribution of each core component in Janus's Harness Synthesizer (§3.2). We reuse the RQ1 setup and metrics on Poirot-44 (44 apps, 21,386 JNI methods). We build four ablation variants, each disabling Janus's explicit workflow at a single stage and instead letting the LLM agent handle that stage unguided, isolating the contribution of each component relative to free-form LLM behavior. We also evaluated an out-of-the-box agent configuration (OpenCode+GLM-5 equipped with default file-inspection and shell tools, without Janus's synthesis workflows, structure-aware retrieval, component memory, or template-based harness validation). Under this setup,

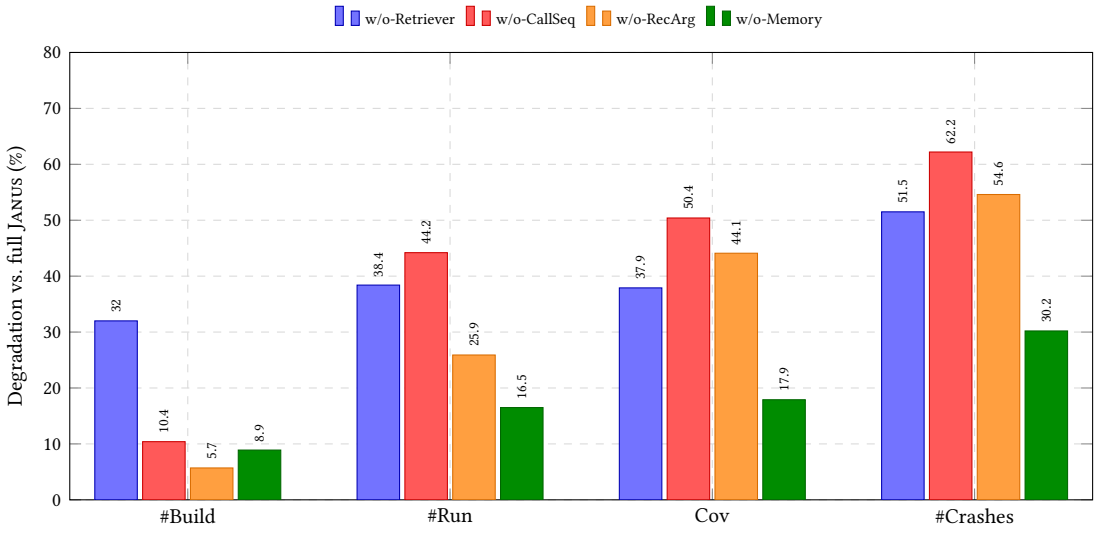


Fig. 5. Component ablation: *degradation* (%) relative to full JANUS.

the agent failed to produce buildable harnesses from the decompiled APK artifacts; we therefore omit it from the quantitative comparison and focus on the controlled ablations above. The four ablation variants are:

- ***w/o-Retriever.*** Disables the structure-aware retriever (§3.2.1); the agent explores decompiled artifacts via textual *grep* only, without JANUS's cross-language retriever.
- ***w/o-CallSeq.*** Disables the dependency-driven call-sequence extractor (§3.2.2); the agent decides on its own how to set up the runtime state of the target method.
- ***w/o-RecArg.*** Disables the recursive argument constructor (§3.2.3); the agent synthesizes argument code freely.
- ***w/o-Memory.*** Disables the memory (§3.2.4); verified call sequences and constructors are no longer reused.

5.2.2 ***Result.*** Disabling any component triggers a distinct regression (Fig. 5). These results show that each component contributes to the performance of the complete JANUS configuration, with degradation patterns consistent with its intended role.

(*i*) **Retriever underpins buildability.** Restricted to textual *grep*, the agent struggles to locate JNI entries, type definitions, and call contexts, so harnesses are built on incomplete code understanding. This produces the largest #Build drop (−32.0%) and cascades to a substantial coverage drop (bitmap 16.9% → 10.5%), establishing structure-aware retrieval as the foundation for all later synthesis steps. (*ii*) **CallSeq bridges *build* and *execution*.** Without prerequisite calls, harnesses still build (#Build −10.4%, a modest drop) but crash due to uninitialized runtime state before reaching the target. The run/build ratio collapses from 84.0% to 52.3%, the lowest among all variants, confirming CallSeq as the critical link from a *buildable* harness to a *runnable* one. (*iii*) **RecArg governs exploration depth.** Free-form arguments are typically type-correct but semantically shallow (null objects, zero-length buffers). Buildability is barely affected (−5.7%), yet fuzzing stalls at native input-validation prologues: coverage drops to 9.4%. Argument legality and completeness are thus prerequisites for penetrating into deep native logic. (*iv*) **Memory sustains quality at scale.** Reuse is not required to get harnesses off the ground, making this variant the mildest (Build −8.9%, Run −16.5%).

Table 2. Effectiveness comparison on VENDOR-20 (best in **bold**).

	Janus				Atlas++				Poirot			
App	#Build	#Run	Cov	#Crashes	#Build	#Run	Cov	#Crashes	#Build	#Run	Cov	#Crashes
backuprestore	**46**	**42**	**14.2**	**4**	11	3	2.1	0	7	7	1.8	0
bbk.theme	**213**	**64**	**18.5**	**5**	64	22	2.7	4	32	18	2.0	0
copilot	**652**	**168**	**24.6**	**28**	88	11	9.8	0	79	38	6.4	22
extraphoto	**43**	**31**	**22.4**	**9**	3	0	0.0	0	13	9	3.4	7
fileexplorer	**47**	**46**	**20.0**	**3**	**47**	16	3.1	2	9	9	2.1	0
gallery3d	**2,622**	**540**	**23.8**	**42**	710	78	2.0	18	365	104	3.2	0
heytap.themestore	**1,156**	**215**	**13.5**	**16**	280	18	1.7	0	154	42	2.3	0
market	**687**	**176**	**15.2**	**14**	152	34	2.6	2	98	29	2.9	9
mediaeditor	**523**	**194**	**26.4**	**48**	126	38	5.9	0	78	34	4.1	19
miui.gallery	**212**	**82**	**25.1**	**15**	58	19	2.9	8	36	16	2.2	0
mms	21	**19**	**6.4**	**1**	**23**	2	0.6	0	4	3	1.1	0
music	**687**	**218**	**21.3**	**64**	112	7	3.8	1	86	36	4.5	8
ocrscanner	**178**	**68**	**11.2**	**7**	49	8	2.5	4	24	12	2.8	1
oplus.themestore	**287**	**54**	**11.8**	**3**	96	5	1.8	0	58	8	2.4	0
pictorial	**175**	**62**	**10.9**	**8**	14	0	0.0	0	20	11	1.5	0
reader	**8**	**7**	**9.5**	**2**	**8**	0	0.0	0	1	0	0.0	0
scanner	**18**	**18**	**17.6**	**6**	6	4	2.3	1	5	5	2.6	4
thememanager	**195**	**64**	**18.9**	**5**	48	15	2.2	3	26	12	1.6	0
video	**89**	**48**	**29.8**	**26**	16	8	11.2	0	18	11	6.2	14
wallpapers	**194**	**58**	**8.2**	0	58	11	2.0	0	28	14	2.2	0
Total	**8,053**	**2,174**	**17.5**	**306**	1,969	299	3.0	43	1,141	418	2.8	84

Note: Columns: **#Build**: harnesses that successfully compile, link, and package (no guarantee of valid execution). **#Run**: harnesses that execute as valid fuzzing targets, i.e., initialize the runtime, load the target native library, consume inputs by executing the target function, and produce growing coverage. **Cov (%)**: AFL++ bitmap coverage. Cov is measured only over the #Run harnesses. **#Crashes**: unique crashes from AFL's deduplication pipeline. The last row gives column totals (mean across apps for Cov). JANUS's improvements are statistically significant (paired Wilcoxon, $p < 0.05$).

Its value is *amplification*: without it, similar methods are re-derived from scratch (multiplying error chances), and high-coverage patterns cannot transfer across related methods, shrinking the aggregate coverage footprint.

Answer to RQ2: LLM agents are not a silver bullet. JANUS introduces key innovations: code retriever, call-sequence extractor, argument constructor, and memory manager, all contributing to its superior performance.

5.3 RQ3: Unknown Bug Discovery

5.3.1 ***Setup***. RQ3 evaluates JANUS's ability to discover unknown vulnerabilities in real-world, security-sensitive apps using the Vendor-20 dataset (described in §5). We extract 11,172 JNI methods from these apps and exclude 1,845 methods following the pre-filtering criteria in §5.1, leaving 9,327 JNI targets. We subject these targets to the same harness-generation pipeline as in RQ1. For each runnable harness, we allocate a fuzzing budget of up to 12 hours with early termination upon coverage saturation (i.e., when no new branches are discovered within a 1-hour window [13, 21, 35, 39, 40]).

5.3.2 ***Result***. JANUS successfully builds 8,053 harnesses from the 9,327 attempted JNI targets (86.3%), of which 2,174 satisfy the #Run criterion defined in RQ1 (§5.1; 27.0% of the built harnesses, outperforming Atlas++ and POIROT by 7.3× and 5.2×). Compared with POIROT-44, the lower #Run rate across all three tools on VENDOR-20 stems from two domain traits: many vendor-specific JNI methods depend on proprietary ROM services or hardware contexts unavailable in standard emulators (§6.2.1), while numerous thin getter/setter JNI bridges in large apps (e.g., `gallery3d`) yield flat coverage and are conservatively excluded by our strict #Run check (§3.3.2). Across all evaluated APKs, JANUS produces 2,535 raw crashing inputs in total. We first deduplicate them based on the crash signal, the target JNI function, the crashing stage, the faulting Program Counter, and the native backtrace, yielding 306 distinct crash groups. We then

Table 3. Native bugs in vendor apps.

Vendor	APP	Library	CWE Type
Vivo	Theme Process	`lib*Pro*.so`	`CWE-787`: *OOB write* ↪ unchecked field write
Vivo	Theme Process	`lib*Pro*.so`	`CWE-502`: *deserialization* ↪ object-state corruption
Xiaomi	Theme Manager	`lib*_nati*.so`	`CWE-787`: *OOB write* ↪ unchecked index write
OnePlus	Reader	`libmo*.so`	`CWE-190`: *integer overflow* ↪ unchecked size math
OPPO	Reader	`libz*bi.so`	`CWE-190`: *integer overflow* ↪ parser-state corruption
OPPO	Video Player	`lib*media*.so`	`CWE-787`: *OOB write* ↪ decode-state corruption

manually triage these groups, filtering out crashes that are unreachable from attacker-controllable production inputs. Finally, root-cause and reachability analyses confirm 6 vulnerabilities, as shown in Table 3. In our evaluation, neither Atlas++ nor Poirot detected any of the six vulnerabilities identified by Janus.

We have responsibly disclosed these vulnerabilities to the corresponding vendors, accompanied by manually validated bug reports, harness artifacts, and crash samples generated by Janus [27]. At the time of submission, one OPPO vulnerability (internal tracking ID: OSRC-2026-2112960957058781184) has been confirmed by the vendor as a zero-day vulnerability with a bug bounty awarded. Additionally, one OPPO vulnerability and one OnePlus vulnerability have been acknowledged (internal tracking IDs: OSRC-2026-2115741462279884800 and OSRC-2026-2115777492253020160) and are currently under vendor investigation. The remaining three reports are still awaiting triage due to vendor backlogs.

Case Study: Heap Out-of-Bounds Write in an Embedded Video Player. Janus uncovered a heap out-of-bounds write in the bundled native media decoder (`lib*media*.so`, invoked via its JNI bridge `lib*JNI*.so`) of a preinstalled video player. When decoding a crafted AVI file containing a video stream, the progressive 2-D reconstruction path validates the header-declared `lowpass_width` only against its internal band allocation, given by `ALIGN(coded_width, 64)/2`, failing to check it against the actual output image-plane width and buffer size. During horizontal filtering, the decoder writes $2 \times$ `lowpass_width` 16-bit samples per row along the plane line stride; for non-block-aligned coded widths, each row spills past the row boundary and the final row writes decoded pixel data beyond the output plane allocation into adjacent heap memory. This corrupts live heap objects and allocator metadata (e.g., Skia rendering pointers, frame-data pointers, or Bionic tagged pointers) and causes fatal process crashes (`SIGSEGV` during rendering or `SIGABRT` during memory deallocation), which we confirmed on a physical OPPO device using multi-frame samples. In practice, an attacker can trigger this vulnerability remotely by delivering a crafted video file through messaging, downloads, or web links, or locally without permissions by passing a media URI to the player's exported, permission-free `ACTION_VIEW` playback activity.

Case Study: Out-of-Bounds Access in Vendor Theme Manager. To illustrate Janus in a real deployment, we examine an out-of-bounds write that Janus surfaced in a native rendering library (lib*_nati*.so) of a privileged theme-management app (Theme Manager) on Xiaomi flagship devices. The triggering harness first issues a sequence of precursor calls to establish the renderer's uniform-buffer state, then mutates the two index arguments passed to the vulnerable function while enforcing the array-length constraints required by the JNI byte-array interface.

The app runs with system privileges and renders user-supplied visual scenes packaged in a custom ZIP-based archive that bundles scene descriptors and assets. The native function derives its write target by multiplying a caller-supplied instance index by a fixed byte stride, but never bounds-checks the index; an oversized index drives the write past the allocated buffer into adjacent heap memory. Since both the write offset (the index) and the written value (a descriptor attribute) are attacker-controlled, the defect constitutes a controllable relative heap-write primitive, remotely reachable by luring a victim into applying a crafted scene file. We also reproduced the crash associated with this vulnerability on a physical Xiaomi device.

Answer to RQ3: Janus effectively discovers previously unknown vulnerabilities in real-world Android apps, confirming 6 vulnerabilities across 4 major vendors.

6 Discussion

6.1 Choice of Base Model

Choosing a backbone for Janus requires balancing long-horizon tool use, harness quality, inference cost, and reproducibility. We tested open-weight models, including DeepSeek-V3.2 [36], Kimi-K2.5 [60], Qwen-3.5 [61], and GLM-5 [72], as well as proprietary models, including GPT-5.2 [48] and Claude Opus 4.5 [1].

To substantiate our choice, we conducted a controlled experiment that varied *only the underlying LLM*, while keeping the Janus workflow, tools, budget, and evaluation pipeline identical. We randomly sampled five apps from Poirot-44, covering 225 JNI methods in total. Table 4 reports buildable harnesses (#Build) and runnable harnesses (#Run), using the same definitions as in RQ1.

• **Sustaining the Agentic Workflow.** In this evaluation, DeepSeek-V3.2, Kimi-K2.5, and Qwen-3.5 synthesized 6, 31, and 40 buildable harnesses, respectively, but only 0, 9, and 19 of them were runnable. When executing long-horizon reasoning, these models frequently violated the tool-calling protocol or terminated prematurely before resolving runtime JNI constraints. These results indicate that, across the tested models and apps, generating syntactically buildable code alone does not guarantee that a backbone LLM can reliably sustain Janus's multi-step agentic workflow.

• **Harness Quality Relative to Frontier Models.** GLM-5 produced 99 buildable and 84 runnable harnesses, compared with 104 and 93 for Claude Opus 4.5, the strongest backbone by both aggregate counts in this subset. Thus, GLM-5 recovered 95.2% (99/104) of Opus 4.5's buildable harness count and 90.3% (84/93) of its runnable harness count. GPT-5.2 produced 102 buildable and 81 runnable harnesses. These results show that GLM-5 approaches the frontier models in harness generation on this subset, while mitigating the severe workflow failures observed with the other tested open-weight backbones.

• **Cost and Reproducibility Trade-Off.** The broader cost comparison concerns the full Poirot-44 workload rather than this five-app experiment: the Opus 4.5 cost is $7,225.8, approximately 5.6× the GLM-5 API cost of $1,292.6. As reported in RQ1, GLM-5 costs approximately $0.21 per runnable harness. Its open weights also facilitate independent deployment and inspection, whereas the closed nature of the proprietary models limits reproducibility. Together, its

Table 4. Backbone model comparison on five randomly sampled apps from Poirot-44 (225 JNI methods).

Package	DeepSeek-V3.2	Kimi-K2.5	Qwen-3.5	GLM-5	Claude Opus 4.5	GPT-5.2
(a) Buildable harnesses (#Build)						
vectorunit	2	8	5	16	18	16
evernote	1	4	3	20	20	20
superbros	0	7	12	23	26	22
billiards.pool	0	3	13	26	26	30
talkinghank	3	9	7	14	14	14
Total	6	31	40	99	104	102
(b) Runnable harnesses (#Run)						
vectorunit	0	3	0	14	18	15
evernote	0	1	1	19	20	19
superbros	0	3	10	16	18	15
billiards.pool	0	0	7	21	23	18
talkinghank	0	2	1	14	14	14
Total	0	9	19	84	93	81

near-frontier harness counts on the sampled apps, robust instruction following, open weights, and low cost justify GLM-5 as Janus's default backbone.

We expect continued improvements in open-weight models to further narrow the observed gap and improve cost-effectiveness, making open, reproducible, and affordable backbones increasingly attractive for agentic harness synthesis.

6.2 Limitations and Threats to Validity

6.2.1 ***Limitations.*** Janus's primary limitations stem from the following aspects, which also highlight potential avenues for future research.

• **Concurrent and Asynchronous Dependencies.** Janus currently focuses on resolving explicit and implicit dependencies to reconstruct the JNI execution context, but is inherently based on a linear execution model. However, Android apps frequently involve complex asynchronous mechanisms (e.g., coroutines) and concurrent mechanisms (e.g., multi-threading). Janus may fail to satisfy state dependencies established by asynchronous calls, and may miss concurrency-related vulnerabilities in highly parallelized native libraries.

• **Framework and System Service Interactions.** Janus currently focuses on modeling the JNI call paths within the app itself, but does not account for deep interactions with the Android (e.g., Binder). Such interactions require complex real-time runtime contexts; consequently, for native libraries that rely heavily on such system interactions, Janus may fail to generate harnesses.

6.2.2 ***Threats to Validity.*** The primary threat to validity stems from the generalizability of our evaluation datasets. Poirot-44 represents a traditional category of Android commercial apps that run with normal user-level permissions. Evaluating solely on such apps may limit the generalizability of our findings to other apps that differ in terms of

provenance, permission levels, and native library characteristics. To mitigate this threat, we introduce Vendor-20, a distinct category of vendor pre-installed apps granted system-level permissions, which exhibits different library traits and access privileges. While these two datasets enrich the diversity of our evaluation, other categories of Android apps may remain uncovered; we leave a broader evaluation across a wider spectrum of app types to future work.

7 Related Work

7.1 Automated Harness Generation for Libraries

To reduce the cost of manually writing fuzzing harnesses, a series of recent works have proposed methods for automatically synthesizing harnesses [12, 16, 23, 25, 28, 30, 56, 57, 74, 76]. Specifically, OGHarn [56] splices candidate C-API harnesses based on header files and produces harnesses through filtering and mutation. Some efforts also target cross-language harness synthesis, such as SnakeCharmer [57], which recovers interface types and exception semantics for Python ↔ C/C++ hybrid libraries and generates harnesses accordingly. More recently, studies have begun to delegate harness synthesis tasks to LLMs and LLM-driven agents [15, 18, 33, 38, 42, 45, 55, 65, 67, 69, 70, 73, 75, 77], demonstrating impressive results in extracting API usage knowledge and generating harnesses. While **source-level LLM pipelines** (e.g., LLM4FDG [75] and CKGFuzzer [67]) and **agentic approaches** such as HarnessAgent [69] effectively synthesize harnesses for open-source C/C++ projects by querying source trees (e.g., headers, type definitions, and call graphs) and routing compilation errors, **they do not address two cross-language challenges inherent to closed-source Android native libraries**. (*i*) unlike C/C++ targets whose API dependencies and state transitions reside within a unified source codebase, Android native libraries are stripped `.so` binaries driven by a managed Java host; valid execution requires recovering app-specific prerequisite calls and tracking explicit value and implicit state bindings across the Java-to-native boundary. (*ii*) whereas C/C++ harnesses instantiate argument types defined directly in native headers, JNI targets accept opaque object references (`jobject`) whose valid construction paths reside in decompiled Java code while their actual field consumption occurs in native code. Synthesizing valid arguments therefore requires reconciling native-side field-access demands with app-side Java construction paths to recursively assemble composite objects.

7.2 Android/JNI Vulnerability Detection

Existing Android vulnerability detection research largely focuses on the Java/Kotlin layer, with limited coverage of native-code security risks [3, 4, 11, 14, 19, 31, 32, 43, 44, 54, 64, 66, 68]. Cross-language static analyses such as NativeSummary [63] extract JNI summaries from native binaries for bytecode analyzers, but target information-flow tracking rather than memory corruption. In terms of Android fuzzing studies: FANS [37] synthesizes harnesses from Android system-service source code but assumes source availability; JNIFuzzer [52] fuzzes JNI methods yet lacks Java-side context modeling. Atlas [66] and Poirot [19] are the closest systems for closed-source native libraries; both infer app-specific JNI usage through static program analysis. Both remain constrained by the precision-scalability trade-off, and neither recovers construction knowledge for complex composite objects in real-world applications.

7.3 Cross-Language Analysis of Android Applications

Prior research [14, 54, 63, 64, 68] has explored static cross-language program analysis across the Java and native boundaries, alongside general-purpose JNI analysis frameworks such as AXA [53] and Confluence [51]. For instance, JN-SAF [64] and NDroid [68] track inter-language data and taint flows, JuCify [54] unifies Java and native intermediate

representations to construct cross-language call graphs (CGs), and NativeSummary [63] extracts native binary summaries to assist bytecode analyzers.

However, these static cross-language analyzers cannot be directly applied to synthesize fuzzing harnesses for closed-source Android native libraries due to three fundamental distinctions. (*i*) **Recovering cross-language call relations does not equate to executable harness synthesis**. Existing tools primarily focus on structural reachability [14] or information-flow tracking [63, 64], rather than on synthesizing standalone C/C++ driver code that reconstructs implicit native-side runtime states or recursively instantiates complex composite Java arguments via reflective JNI APIs. (*ii*) **General-purpose Java-native analysis frameworks face ecosystem incompatibilities on Android**. Frameworks such as AXA rely on Java bytecode analyzers (e.g., OPAL) tailored for standard Java bytecode rather than Android DEX bytecode and lifecycle-driven app components, rendering them unsuitable for analyzing commercial Android APKs. (*iii*) **CG-based approaches suffer from a severe precision-scalability trade-off on modern apps**. Heavyweight cross-language CG construction and symbolic analysis (e.g., JuCify [54], Atlas [66]) frequently time out or yield empty results on large-scale, obfuscated apps. Conversely, lightweight Java-side CG approximations (e.g., Poirot [19]) miss native-side state transitions and cross-callback initialization sequences (such as global states initialized in lifecycle entry points outside the target's direct call chain), omitting critical setup and argument construction and thereby inducing false-positive crashes from corrupted runtime states.

To mitigate the limitations of conventional cross-language program analysis in recovering the JNI usage knowledge required for **executable harness generation**, Janus introduces dependency-driven call-sequence extraction to reconstruct the app-specific prerequisite context needed for valid JNI execution. It further exploits semantic relationships among parameters to recursively construct complex composite arguments, enabling the generated harnesses to effectively exercise closed-source Android native libraries.

8 Conclusion

In this paper, we propose Janus, the first agentic approach to synthesizing JNI-aware fuzzing harnesses for Android native libraries. Grounded in structure-aware cross-language code retrieval, Janus iteratively resolves both explicit and implicit call dependencies to reconstruct app-specific invocation contexts, and recursively assembles complex composite argument objects in a bottom-up manner, supported by harness memory and validation-guided iterative refinement. Extensive evaluation on real-world commercial and vendor pre-installed apps demonstrates that Janus outperforms state-of-the-art baselines, yielding more valid harnesses (7.4×/5.9× more runnable harnesses) and uncovering previously unknown vulnerabilities. Our work lays the foundation for agentic cross-language fuzzing harness generation in the Android ecosystem. Beyond standalone harness synthesis, Janus sheds light on downstream research fields for Android native code security. Many current and future Android cross-language security vetting and dynamic analysis initiatives can be enhanced by incorporating Janus's insights.

9 Data Availability

To facilitate reproducibility and foster future research, we publicly release our prototype and artifacts [27].

Acknowledgments

This work was supported by the National Key R&D Program of China (Grant No.2024YFE0203800), the National Natural Science Foundation of China (No. 62472309), and the grant from Zhongguancun Academy (Grant No. 20240302).

References

[1] 2025. Introducing Claude Opus 4.5. [Online]. Available: https://www.anthropic.com/news/claude-opus-4-5. Accessed: Jun. 27, 2026.

[2] 2026. Agent Skills. [Online]. Available: https://opencode.ai/docs/skills/. Accessed: Oct. 25, 2026.

[3] Yasemin Acar, Michael Backes, Sven Bugiel, Sascha Fahl, Patrick McDaniel, and Matthew Smith. 2016. Sok: Lessons learned from android security research for appified software platforms. In *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 433–451.

[4] Sumaya Almanee, Arda Ünal, Mathias Payer, and Joshua Garcia. 2021. Too quiet in the library: An empirical study of security updates in android apps' native code. In *2021 ieee/acm 43rd international conference on software engineering (icse)*. IEEE, 1347–1359.

[5] Android Developers. [n. d.]. JNI Tips. [Online]. Available: https://developer.android.com/ndk/guides/jni-tips. Last updated: Mar. 6, 2026. Accessed: Jun. 30, 2026.

[6] Android Open Source Project. [n. d.]. Memory Safety. [Online]. Available: https://source.android.com/docs/security/test/memory-safety. Accessed: Jun. 27, 2026.

[7] Android Open Source Project. 2026. Android Security Bulletin—February 2026. [Online]. Available: https://source.android.com/docs/security/bulletin/2026/2026-02-01. Accessed: Jun. 24, 2026.

[8] Anomaly Innovations. [n. d.]. OpenCode: The Open Source Coding Agent. [Online]. Available: https://github.com/anomalyco/opencode. Accessed: Jun. 21, 2026.

[9] Anthropic. [n. d.]. Claude Code: Anthropic's Agentic Coding System. [Online]. Available: https://www.anthropic.com/product/claude-code. Accessed: Jun. 27, 2026.

[10] AtlasPlusPlus. [n. d.]. Atlas++: Automating Cross-Language Fuzzing on Android Closed-Source Libraries. [Online]. Available: https://github.com/AtlasPlusPlus/Atlas-plusplus. Accessed: Jun. 21, 2026.

[11] Michael Auer, Andreas Stahlbauer, and Gordon Fraser. 2023. Android fuzzing: Balancing user-inputs and intents. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 37–48.

[12] Domagoj Babić, Stefan Bucur, Yaohui Chen, Franjo Ivančić, Tim King, Markus Kusano, Caroline Lemieux, László Szekeres, and Wei Wang. 2019. Fudge: fuzz driver generation at scale. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 975–985.

[13] Marcel Böhme and Brandon Falk. 2020. Fuzzing: On the Exponential Cost of Vulnerability Discovery. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 749–760. doi:10.1145/3368089.3409729

[14] Luca Borzacchiello, Emilio Coppa, Davide Maiorca, Andrea Columbu, Camil Demetrescu, and Giorgio Giacinto. 2022. Reach me if you can: On native vulnerability reachability in android apps. In *European Symposium on Research in Computer Security*. Springer, 701–722.

[15] Chuyang Chen, Brendan Dolan-Gavitt, and Zhiqiang Lin. 2025. {ELFuzz}: Efficient Input Generation via {LLM-driven} Synthesis Over Fuzzer Space. In *34th USENIX Security Symposium (USENIX Security 25)*. 6279–6298.

[16] Peng Chen, Yuxuan Xie, Yunlong Lyu, Yuxiao Wang, and Hao Chen. 2023. Hopper: Interpretative fuzzing for libraries. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1600–1614.

[17] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, and Ion Stoica. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 8359–8388.

[18] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.

[19] Luca Di Bartolomeo, Philipp Mao, Yu-Jye Tung, Jessy Ayala, Samuele Doria, Paolo Celada, Marcel Busch, Joshua Garcia, Eleonora Losiouk, and Mathias Payer. 2025. Hercules Droidot and the murder on the {JNI} Express. In *34th USENIX Security Symposium (USENIX Security 25)*. 3257–3275.

[20] Eclipse JDTLS. [n. d.]. Eclipse JDT Language Server. [Online]. Available: https://github.com/eclipse-jdtls/eclipse.jdt.ls. Accessed: Jun. 21, 2026.

[21] Wentao Gao, Thuan Pham, Dongge Liu, Oliver Chang, Toby Murray, and Benjamin I. P. Rubinstein. 2023. Beyond the Coverage Plateau: A Comprehensive Study of Fuzz Blockers. In *Proceedings of the 2nd International Fuzzing Workshop (FUZZING)*. 35–43. doi:10.1145/3605157.3605173

[22] Google Project Zero. 2026. A 0-click exploit chain for the Pixel 10: When a Door Closes, a Window Opens. [Online]. Available: https://projectzero.google/2026/05/pixel-10-exploit.html. Seth Jenkins. Accessed: Jun. 21, 2026.

[23] Harrison Green and Thanassis Avgerinos. 2022. Graphfuzz: Library api fuzzing with lifetime-aware dataflow graphs. In *Proceedings of the 44th International Conference on Software Engineering*. 1070–1081.

[24] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716* (2024).

[25] Kyriakos Ispoglou, Daniel Austin, Vishwath Mohan, and Mathias Payer. 2020. {FuzzGen}: Automatic fuzzer generation. In *29th USENIX Security Symposium (USENIX Security 20)*. 2271–2287.

[26] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

[27] JANUS. [n. d.]. [Online]. Available: https://github.com/Project-Z5O1/Janus.

[28] Bokdeuk Jeong, Joonun Jang, Hayoon Yi, Jiin Moon, Junsik Kim, Intae Jeon, Taesoo Kim, WooChul Shim, and Yong Ho Hwang. 2023. Utopia: Automatic generation of fuzz driver using unit tests. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2676–2692.

[29] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations (ICLR)*.

[30] Jinho Jung, Stephen Tong, Hong Hu, Jungwon Lim, Yonghwi Jin, and Taesoo Kim. 2021. Winnie: Fuzzing windows applications with harness synthesis and fast cloning. In *Proceedings of the 2021 Network and Distributed System Security Symposium (NDSS 2021)*.

[31] Wang Kai, Zhang Yuqing, Liu Qixu, and Fan Dan. 2015. A fuzzing test for dynamic vulnerability detection on Android Binder mechanism. In *2015 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 709–710.

[32] Anatoli Kalysch, Mark Deutel, and Tilo Müller. 2020. Template-based Android inter process communication fuzzing. In *Proceedings of the 15th International Conference on Availability, Reliability and Security*. 1–6.

[33] Taesoo Kim, HyungSeok Han, Soyeon Park, Dae R Jeong, Dohyeok Kim, Dongkwan Kim, Eunsoo Kim, Jiho Kim, Joshua Wang, Kangsu Kim, et al. 2025. ATLANTIS: AI-driven Threat Localization, Analysis, and Triage Intelligence System. *arXiv preprint arXiv:2509.14589* (2025).

[34] Eric Le Guevel. 2023. Android Greybox Fuzzing with AFL++ Frida Mode. [Online]. Available: https://blog.quarkslab.com/android-greybox-fuzzing-with-afl-frida-mode.html. Accessed: Jun. 21, 2026.

[35] Stephan Lipp, Daniel Elsner, Severin Kacianka, Alexander Pretschner, Marcel Böhme, and Sebastian Banescu. 2023. Green Fuzzing: A Saturation-Based Stopping Criterion using Vulnerability Prediction. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 127–139. doi:10.1145/3597926.3598043

[36] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556* (2025).

[37] Baozheng Liu, Chao Zhang, Guang Gong, Yishun Zeng, Haifeng Ruan, and Jianwei Zhuge. 2020. {FANS}: Fuzzing android native system services via automated interface analysis. In *29th USENIX Security Symposium (USENIX Security 20)*. 307–323.

[38] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Zhilin Tian, Yuekai Huang, Jun Hu, and Qing Wang. 2024. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model. In *Proceedings of the IEEE/ACM 46th International conference on software engineering*. 1–12.

[39] Danushka Liyanage, Marcel Böhme, Chakkrit Tantithamthavorn, and Stephan Lipp. 2023. Reachable Coverage: Estimating Saturation in Fuzzing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 371–383. doi:10.1109/ICSE48619.2023.00042

[40] Danushka Liyanage, Seongmin Lee, Chakkrit Tantithamthavorn, and Marcel Böhme. 2024. Extrapolating Coverage Rate in Greybox Fuzzing. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–12. doi:10.1145/3597503.3639199

[41] LLVM Project. [n. d.]. clangd: Teach Your Editor C++. [Online]. Available: https://clangd.llvm.org/. Accessed: Jun. 21, 2026.

[42] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. Prompt fuzzing for fuzz driver generation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3793–3807.

[43] Amiya K Maji, Fahad A Arshad, Saurabh Bagchi, and Jan S Rellermeyer. 2012. An empirical study of the robustness of inter-component communication in android. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*. IEEE, 1–12.

[44] Philipp Mao, Marcel Busch, and Mathias Payer. 2025. {NASS}: Fuzzing All Native Android System Services with Interface Awareness and Coverage. In *34th USENIX Security Symposium (USENIX Security 25)*. 4225–4243.

[45] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large Language Model guided Protocol Fuzzing.. In *NDSS*.

[46] National Security Agency. 2019. Ghidra: A Software Reverse Engineering (SRE) Suite of Tools. https://ghidra-sre.org/. Accessed: 2026-10-27.

[47] OpenAI. [n. d.]. Codex CLI. [Online]. Available: https://learn.chatgpt.com/docs/codex/cli. Accessed: Oct. 27, 2026.

[48] OpenAI. 2025. Introducing GPT-5.2. [Online]. Available: https://openai.com/index/introducing-gpt-5-2/. Accessed: Jun. 27, 2026.

[49] OpenCode. 2026. Tools. [Online]. Available: https://opencode.ai/docs/tools/. Accessed: Oct. 25, 2026.

[50] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).

[51] Jyoti Prakash, Abhishek Tiwari, and Christian Hammer. 2025. Modular unification of unilingual pointer analyses to multilingual FFI-based programs. *Science of Computer Programming* 243 (2025), 103278. doi:10.1016/j.scico.2025.103278

[52] Claudio Rizzo. 2020. *Static Flow Analysis for Hybrid and Native Android Applications*. Ph. D. Dissertation. Royal Holloway, University of London.

[53] Tobias Roth, Julius Näumann, Dominik Helm, Sven Keidel, and Mira Mezini. 2024. AXA: Cross-Language Analysis through Integration of Single-Language Analyses. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1195–1205. doi:10.1145/3691620.3696193

[54] Jordan Samhi, Jun Gao, Nadia Daoudi, Pierre Graux, Henri Hoyez, Xiaoyu Sun, Kevin Allix, Tegawendé F Bissyandé, and Jacques Klein. 2022. Jucify: A step towards android code unification for enhanced static analysis. In *Proceedings of the 44th International Conference on Software Engineering*. 1232–1244.

[55] Ze Sheng, Qingxiao Xu, Jianwei Huang, Matthew Woodcock, Heqing Huang, Alastair F Donaldson, Guofei Gu, and Jeff Huang. 2025. All You Need Is A Fuzzing Brain: An LLM-Powered System for Automated Vulnerability Detection and Patching. *arXiv preprint arXiv:2509.07225* (2025).

[56] Gabriel Sherman and Stefan Nagy. 2025. No Harness, No Problem: Oracle-guided Harnessing for Auto-generating C API Fuzzing Harnesses. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 165–177.

[57] GABRIEL SHERMAN and STEFAN NAGY. 2026. SnakeCharmer: Automatic Fuzzing Harness Generation for Pure and Hybrid Python Libraries. (2026).

[58] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. 8634–8652. doi:10.52202/075280-0377

[59] Skylot. 2013. JADX: Dex to Java Decompiler. https://github.com/skylot/jadx. Accessed: 2026-10-27.

[60] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. 2026. Kimi K2. 5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* (2026).

[61] Qwen Team. 2026. Qwen3. 5-omni technical report. *arXiv preprint arXiv:2604.15804* (2026).

[62] Unit 42. 2025. LANDFALL: New Commercial-Grade Android Spyware in Exploit Chain Targeting Samsung Devices. [Online]. Available: https://unit42.paloaltonetworks.com/landfall-is-new-commercial-grade-android-spyware/. Accessed: Jun. 21, 2026.

[63] Jikai Wang and Haoyu Wang. 2024. Nativesummary: Summarizing native binary code for inter-language static analysis of android apps. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis*. 971–982.

[64] Fengguo Wei, Xingwei Lin, Xinming Ou, Ting Chen, and Xiaosong Zhang. 2018. Jn-saf: Precise and efficient ndk/jni-aware inter-language static analysis framework for security vetting of android applications with native code. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1137–1150.

[65] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[66] Hao Xiong, Qinming Dai, Rui Chang, Mingran Qiu, Renxiang Wang, Wenbo Shen, and Yajin Zhou. 2024. Atlas: Automating cross-language fuzzing on android closed-source libraries. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 350–362.

[67] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2025. Ckgfuzzer: Llm-based fuzz driver generation enhanced by code knowledge graph. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 243–254.

[68] Lei Xue, Chenxiong Qian, Hao Zhou, Xiapu Luo, Yajin Zhou, Yuru Shao, and Alvin TS Chan. 2018. Ndroid: Toward tracking information flows across multiple android contexts. *IEEE Transactions on Information Forensics and Security* 14, 3 (2018), 814–828.

[69] Kang Yang, Yunhang Zhang, Zichuan Li, Guanhong Tao, Jun Xu, and Xiaojing Liao. 2025. HarnessAgent: Scaling Automatic Fuzzing Harness Construction with Tool-Augmented LLM Pipelines. *arXiv preprint arXiv:2512.03420* (2025).

[70] Yupeng Yang, Shenglong Yao, Jizhou Chen, and Wenke Lee. 2025. Hybrid Language Processor Fuzzing via {LLM-Based} Constraint Solving. In *34th USENIX Security Symposium (USENIX Security 25)*. 6299–6318.

[71] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

[72] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. 2026. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763* (2026).

[73] Haochen Zeng, Andrew Bao, Jiajun Cheng, and Chengyu Song. 2025. PBFuzz: Agentic Directed Fuzzing for PoV Generation. *arXiv preprint arXiv:2512.04611* (2025).

[74] Cen Zhang, Xingwei Lin, Yuekang Li, Yinxing Xue, Jundong Xie, Hongxu Chen, Xinlei Ying, Jiashui Wang, and Yang Liu. 2021. {APICraft}: Fuzz driver generation for closed-source {SDK} libraries. In *30th USENIX Security Symposium (USENIX Security 21)*. 2811–2828.

[75] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2024. How effective are they? exploring large language model based fuzz driver generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1223–1235.

[76] Mingrui Zhang, Chijin Zhou, Jianzhong Liu, Mingzhe Wang, Jie Liang, Juan Zhu, and Yu Jiang. 2023. Daisy: Effective fuzz driver synthesis with object usage sequence analysis. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 87–98.

[77] Xiaochen Zou, Juefei Pu, Arrdya Srivastav, Jonathan Cox, Zhengchuan Liang, Yuan Tan, Xingyu Li, Yilin Zhu, and Zhiyun Qian. [n. d.]. StepStone: LLM-Based GPU Kernel Driver Fuzzing via User-Space Libraries. ([n. d.]).